

Módulo 14: Ciencia de datos

Contenidos clave

NumPy, Polars y Pandas:

- NumPy: es una librería para trabajar con datos numéricos y arreglos de manera eficiente. Ayuda a hacer operaciones sobre muchos valores sin tener que recorrerlos manualmente con un for.
- Pandas: está orientado a manipular y analizar datos estructurados, especialmente tablas.
- Polars: también permite trabajar con datos tabulares, parecido a Pandas. Está diseñado para ser muy eficiente y aprovechar procesamiento paralelo.

scikit-learn: modelos clásicos: es una de las librerías más utilizadas para Machine Learning clásico. Permite crear modelos como: clasificación, regresión, clustering, árboles de decisión, KNN, regresión logística Random Forest, etc..

Serialización de modelos e inferencia básica: significa utilizar un modelo que ya fue entrenado para obtener una predicción con nuevos datos:

- Entrenamiento: Datos históricos -> entrenamiento -> modelo
- Inferencia: Nuevo dato -> modelo entrenado -> predicción

Objetivos

- Manipular datos y entrenar un modelo simple.

Se cumplió mediante:

- La utilización de pandas para cargar y limpiar el CSV en preprocessing.py.
- Validación de columnas verificando que existan.
- Conversión de tipos numéricos.
- Eliminación de registros inválidos
- Separación de características y objetivo.
- Entrenamiento de un clasificador de regresión logística.
- Uso de pipeline con estandarización.

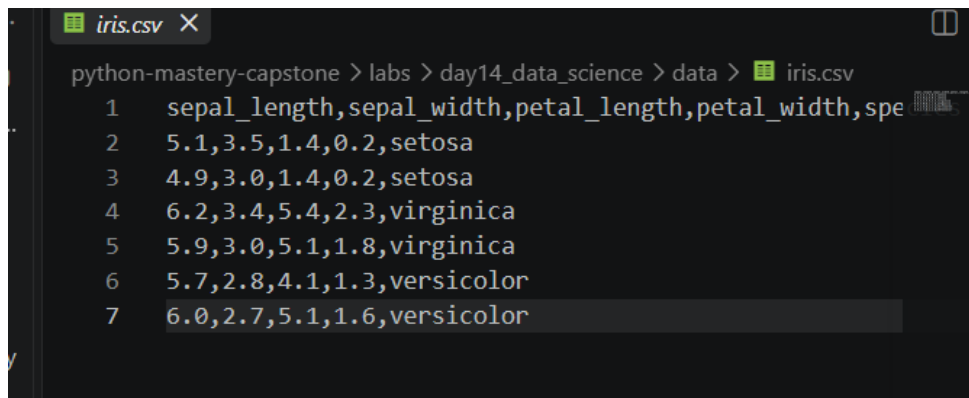
- Exponer inferencia mínima.

Se cumplió mediante [inference.py](#): Carga el modelo entrenado, recibe los valores de una muestra, los convierte al formato esperado, ejecuta la predicción y devuelve la especie estimada.

Laboratorio

- Cargar/limpiar CSV en Pandas.

Se usó un CSV pequeño de clasificación de flores:



```
python-mastery-capstone > labs > day14_data_science > data > iris.csv
1 sepal_length,sepal_width,petal_length,petal_width,spe
2 5.1,3.5,1.4,0.2,setosa
3 4.9,3.0,1.4,0.2,setosa
4 6.2,3.4,5.4,2.3,virginica
5 5.9,3.0,5.1,1.8,virginica
6 5.7,2.8,4.1,1.3,versicolor
7 6.0,2.7,5.1,1.6,versicolor
```

Y se implementó `load_dataset()` para:

- Leer el CSV.
- Validar columnas requeridas.
- Eliminar valores nulos.
- Convertir las características a valores numéricos.
- Eliminar filas con datos inválidos.

- Entrenar un clasificador.

Se entrenó un pipeline con: `StandardScaler` (normaliza las variables) y `LogisticRegression` (clasifica las flores); además de una división estratificada entre entrenamiento y prueba. Carga y limpia el dataset, utiliza `stratify` para conservar la proporción de clase.

- Guardar con `joblib` y probar inferencia.

Se implementó la serialización con `joblib.dump(model, output_path)`, se cargó el modelo y se probó la predicción de una nueva muestra mediante `model.predict(dataframe)`. El modelo se guarda completo, incluyendo el escalador y el clasificador, para que la inferencia use exactamente el mismo procesamiento de entrenamiento.

[Inference.py](#) realiza predicciones:

- `load_model()` carga un modelo serializado.
- `predict()` recibe las características de una nueva flor.
- Convierte los valores a un `DataFrame`.
- Usa el pipeline para predecir la especie.
- Devuelve la clase como texto.

Las pruebas cargan o utilizan el modelo entrenado y verifican que una muestra produzca una clase válida: setosa, versicolor, virginica.

Pruebas. Se validaron:

- Carga correcta del dataset.
- Limpieza de datos.
- Separación de características y objetivo.
- Entrenamiento.
- Generación del archivo `.joblib`.

- Inferencia de clases válidas.

Prueba de inferencia con sample:

```
sample = {  
    "sepal_length": 5.1,  
    "sepal_width": 3.5,  
    "petal_length": 1.4,  
    "petal_width": 0.2,  
}
```

Se obtiene:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>  
poetry run python -m labs.day14_data_science.run_inference  
Modelo entrenado y guardado en: labs\day14_data_science\data\iris_model.joblib  
Inferencia ejecutada correctamente  
Entrada: {'sepal_length': 5.1, 'sepal_width': 3.5, 'petal_length': 1.4, 'petal_width': 0.2}  
Especie predicha: setosa  
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Cambiando el sample a:

```
26     sample = {  
27         "sepal_length": 6.0,  
28         "sepal_width": 2.7,  
29         "petal_length": 5.1,  
30         "petal_width": 1.6,  
31     }  
32
```

Se obtiene:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>  
poetry run python -m labs.day14_data_science.run_inference  
Inferencia ejecutada correctamente  
Entrada: {'sepal_length': 6.0, 'sepal_width': 2.7, 'petal_length': 5.1, 'petal_width': 1.6}  
Especie predicha: versicolor  
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Pruebas con pytest:

```
poetry run pytest .\labs\day14_data_science\tests `
>> --cov=labs.day14_data_science `
>> --cov-report=term-missing `
>> --cov-report=html `
>> --cov-fail-under=80
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 3 items

labs\day14_data_science\tests\test_model.py ... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.14.6-final-0 -----

Name                                                    Stmts  Miss  Cover   Missing
-----
labs\day14_data_science\inference.py                   11     1    91%    11
labs\day14_data_science\model.py                      17     0   100%
labs\day14_data_science\preprocessing.py              19     1    95%    26
-----
TOTAL                                                    47     2    96%
Coverage HTML written to dir htmlcov
Required test coverage of 80% reached. Total coverage: 95.74%
===== 3 passed in 2.03s =====
```